Control flow

Control flow

1. `if`/`else` statements

   If condition is `TRUE`, then perform an action.

2. `while`

   Perform an action while the condition remains `TRUE`.

3. `repeat`

   Repeat an action forever.

We're only going to look at `if`/`else`.

`if` / `else`

```
if (CONDITION is TRUE) {
   DO SOMETHING
} else {
   DO SOMETHING ELSE
}
```

`if` / `else`

```
if (CONDITION is TRUE) {
   DO SOMETHING
} else {
   DO SOMETHING ELSE
}
```

```r
hour <- as.numeric(format(Sys.time(), "%H"))
if (hour < 12) {
  print("Good morning")
} else {
  print("Good afternoon")
}
```

```r
if (score >= 50) {
    grade <- "pass"
} else {
    grade <- "fail"
}
```

```r
if (score >= 50) {
    grade <- "pass"
} else {
    grade <- "fail"
}
```

We can have multiple conditions:

```r
if (score >= 70) {
    grade <- "A"
} else if (score >= 60) {
    grade <- "B"
} else if (score >= 50) {
    grade <- "C"
} else {
    grade <- "fail"
}
```

`if` checks a single value

`if` expects a condition of length 1. Give it a vector and you get an error:

```
> scores <- c(45, 72, 58)
> if (scores >= 50) "pass" else "fail"
Error in if (scores >= 50) "pass" else "fail" :
  the condition has length > 1
```

`if` checks a single value

`if` expects a condition of length 1. Give it a vector and you get an error:

```
> scores <- c(45, 72, 58)
> if (scores >= 50) "pass" else "fail"
Error in if (scores >= 50) "pass" else "fail" :
  the condition has length > 1
```

To check every element of a vector, use `ifelse()`:

```
ifelse(CONDITION, VALUE IF TRUE, VALUE
IF FALSE)
```

```
> ifelse(scores >= 50, "pass", "fail")
[1] "fail" "pass" "pass"
```

But don't overuse `else if` — it quickly becomes unreadable.

We'll introduce `if_else`, `case_when` and `recode_values` in Session 2. These are a better option if you have multiple conditions to evaluate.

For example:

```
library(dplyr)

case_when(
  score >= 70 ~ "A",
  score >= 60 ~ "B",
  score >= 50 ~ "C",
  .default = "Fail"
)
```